

Trabalho Prático: Detecção de Bad Smells e Refatoração Segura

Lucas Flor Vilela
Engenharia de Software – 6^a Período

Maio de 2026

1 Introdução

Este relatório apresenta o processo de análise estática, identificação de *Bad Smells* (maus cheiros no código) e refatoração segura aplicada ao serviço de relatórios `ReportGenerator`. O objetivo principal consiste em utilizar ferramentas automatizadas de linting em conjunto com uma suíte de testes unitários para atuar como rede de segurança, provando que melhorias estruturais no *design* de código não alteram o comportamento esperado do sistema.

2 Análise e Identificação de Bad Smells

A análise manual do arquivo original `src/ReportGenerator.js` revelou três problemas clássicos de *design* de código, detalhados a seguir:

- **Método Longo (Long Method):** O método `generateReport` centraliza todas as responsabilidades do fluxo de negócio. Ele realiza a filtragem com base nas permissões do usuário, acumula os valores financeiros e lida com as particularidades de formatação das estruturas de saída (HTML e CSV). Essa falta de coesão fere diretamente o Princípio da Responsabilidade Única (SRP).
- **Duplicação de Código (Code Duplication):** A lógica usada para construir as linhas textuais do formato CSV e as linhas da tabela HTML repete-se em blocos quase idênticos tanto nas condições de `ADMIN` quanto de `USER`. Mudanças estruturais futuras em qualquer layout exigiriam alterações manuais redundantes em múltiplos pontos.
- **Complexidade Ciclométrica e Condicionais Aninhadas:** O uso excessivo de estruturas `if/else` aninhadas dentro do laço de repetição `for` prejudica seriamente a legibilidade do código. A ramificação excessiva torna o fluxo lógico tortuoso e dificulta a introdução de novos formatos de relatório (como PDF ou JSON).

3 Relatório da Ferramenta de Análise Estática

Para complementar a análise estrutural, configurou-se o linter `ESLint` em conjunto com o *plugin* `eslint-plugin-sonarjs`. A ferramenta foi executada utilizando regras estritas de qualidade sobre o arquivo de produção original.

Como observado na Figura 1, o *plugin* acusou duas violações graves:

1. **sonarjs/cognitive-complexity:** A complexidade cognitiva do método foi avaliada em 27 pontos, ultrapassando drasticamente os limites toleráveis. Diferente da complexidade ciclométrica clássica (que conta caminhos matemáticos), a complexidade cognitiva mensura o esforço mental real exigido de um programador para compreender o fluxo textual. O aninhamento severo justifica a alta pontuação.
2. **sonarjs/no-collapsible-if:** Detectou condicionais aninhadas redundantes na linha 44, sugerindo a unificação lógica para reduzir níveis desnecessários de indentação.

```
⊗ $ npx eslint src/

C:\Users\Lucas\Documents\GitHub\bad-smells-js-refactoring\src\ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to
the 5 allowed  sonarjs/cognitive-complexity
  44:14  error  Merge this if statement with the nested one
              sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

Figura 1: Erros de Bad Smell identificados pelo ESLint/SonarJS no arquivo original.

4 Processo de Refatoração (Antes e Depois)

O ponto mais crítico atacado foi o método longo `generateReport`. Utilizou-se predominantemente a técnica de refatoração **Extract Method (Extração de Método)** para delegar as subtarefas a funções privadas menores, isoladas e coesas.

O laço de iteração principal, que antes acumulava lógicas cruzadas de permissão e concatenação de strings complexas, foi reduzido a chamadas semânticas limpas:

```
1 // DEPOIS DA REFATORAÇÃO 0: Fluxo principal simplificado
2 generateReport(reportType, user, items) {
3   const filteredItems = this._filterItemsByUser(user, items);
4   let report = this._generateHeader(reportType, user);
5   let total = 0;
6
7   for (const item of filteredItems) {
8     report += this._generateRow(reportType, user, item);
9     total += item.value;
10  }
11
12  report += this._generateFooter(reportType, total);
13  return report.trim();
14 }
```

Ao isolar os comportamentos nos métodos auxiliares `_filterItemsByUser`, `_generateHeader`, `_generateRow` e `_generateFooter`, o código tornou-se modular e autoexplicativo, eliminando por completo a duplicação lógica e diminuindo os níveis de aninhamento condicional.

5 Validação Final e Rede de Segurança

Para garantir que as modificações internas estruturais não violaram as regras de negócio vigentes do sistema de relatórios, utilizou-se a suíte de testes unitários baseada no **Jest**. O arquivo de testes foi duplicado e apontado para o arquivo refatorado.

Como evidenciado na Figura 2, todos os 10 cenários de teste (divididos entre o comportamento legado e o refatorado) passaram sem falhas, atestando a integridade funcional

```
• $ npm test

> bad-smells-js-refactoring@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js

(node:12400) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS tests/ReportGenerator.refactored.test.js
PASS tests/ReportGenerator.test.js

Test Suites: 2 passed, 2 total
Tests:      10 passed, 10 total
Snapshots:  0 total
Time:       1.129 s
```

Figura 2: Suíte de testes rodando e validando com sucesso os dois arquivos simultaneamente.

do sistema.

Finalmente, submeteu-se o novo arquivo `ReportGenerator.refactored.js` ao escrutínio do ESLint com o *plugin* SonarJS para comprovar a eficácia da refatoração.

```
Lucas@DESKTOP-5CJQ9B2 MINGW64 ~/Documents/GitHub/bad-smells-js-refactoring (main)
• $ npx eslint src/ReportGenerator.refactored.js
```

Figura 3: Execução do linter sobre o arquivo refatorado, sem emissão de alertas ou erros.

A ausência de saídas ou listagens de erros na Figura 3 valida que os problemas de complexidade cognitiva elevada e estruturas colapsáveis de `if` foram integralmente mitigados, resultando em um código em total conformidade com as métricas de qualidade modernas.

6 Conclusão

O desenvolvimento deste projeto evidenciou a extrema importância de manter testes automatizados estruturados como uma **rede de segurança** ativa durante ciclos de melhoria arquitetural. A liberdade de modificar algoritmos complexos sabendo que qualquer desvio lógico seria imediatamente acusado por uma asserção previne a introdução de regressões em ambientes produtivos.

Ademais, a união da análise manual com ferramentas estáticas robustas como o ESLint e SonarJS demonstra que a qualidade de *software* pode ser mensurada e monitorada de forma contínua. A eliminação dos *Bad Smells* reduziu drasticamente o débito técnico do componente de relatórios, simplificando manutenções futuras e facilitando a extensão do ecossistema.